

CS202: Programming Systems

Week 6: Multiple inheritance

CS202 – What will be discussed?

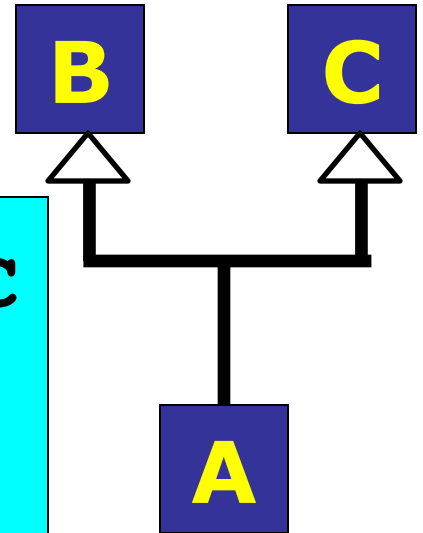
- ☐ Multiple inheritance
- ☐ Diamond problem
- ☐ Virtual inheritance

Multiple inheritance

- When a class has 2 or more direct base classes, it is called ***multiple inheritance***.

- For example

```
class A: public B, public C
{
    ...
};
```



Multiple inheritance

- ❑ Data members and operations from B and C will be inherited to class A similarly to ***single inheritance*** mentioned last time.
- ❑ Virtual functions work as usual

Example

```
class B {  
    void draw();  
};  
class C {  
    void calc();  
};  
class A: public B,  
        public C  
{  
    void doSth();  
};
```

```
void test(A& a)  
{  
    // B::draw()  
    a.draw();  
    // C::calc()  
    a.calc();  
    // A::doSth()  
    a.doSth();  
}
```

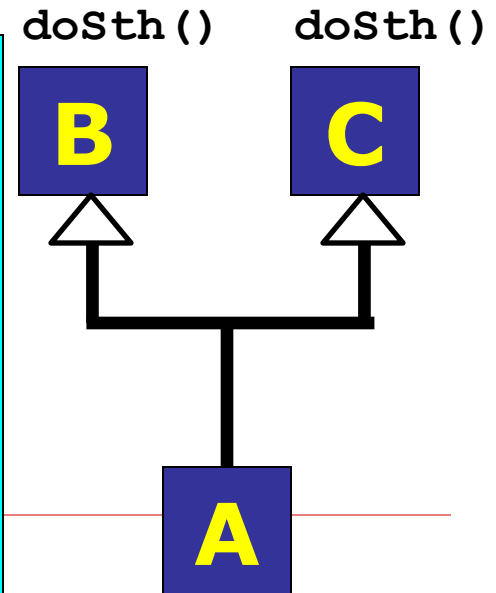
Dynamic binding

```
class B {  
    virtual void draw() = 0;  
};  
class C {  
    virtual void calc() = 0;  
};  
class A: public B, public C  
{  
    void draw(); //override B::draw()  
    void calc(); //override C::calc()  
};
```

Function name clash: ambiguity

- ❑ Overload resolution is not applied across different class scopes. It means function ambiguities from different base classes are not resolved based on function signatures.

```
int main()
{
    A a;
    a.doSth();      //error:ambiguous
    a.B::doSth();  // OK
    a.C::doSth();  // OK
}
```



using keyword

- If the use of the same name in different base classes is deliberately and the user would like to choose the function based on its signature
- **using** declaration can bring the functions into a common scope.

Function name clashes!!!

```
class B {  
    void doSth(int);  
};  
class C {  
    void doSth(double);  
};  
class A: public B, public C {...};  
  
void test() {  
    A a;  
    a.doSth(10); //Error: ambiguous!  
};
```

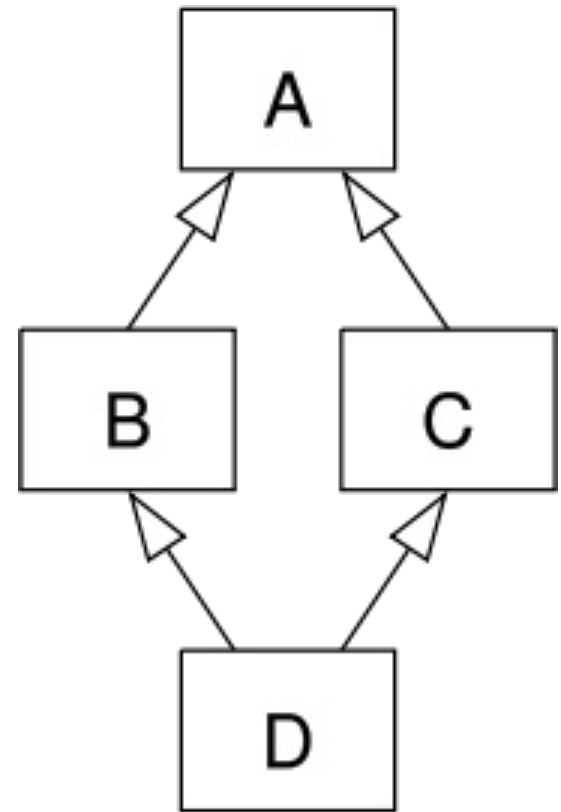
```
class B {  
    void doSth(int);  
};  
  
class C {  
    void doSth(double);  
};  
  
class A: public B, public C {  
    using C::doSth;  
    using B::doSth;  
    void doSth(char);  
};  
  
void test(A& a) {  
    a.doSth(10);    // B::doSth(int)  
    a.doSth('a');  // A::doSth(char)  
    a.doSth(5.2);  // C::doSth(double)  
};
```

Replicated based class

- With the ability of specifying more than one base class, there may be a chance of having the same base class more than once.

Diamond problem!

```
class A { ... };  
class B: public A  
{ ... };  
class C: public A  
{ ... };  
class D: public B,  
        public C  
{ ... };
```



Replicated based class

```
void test(D* p)
{
    p->doSth() ; // error: ambiguous
    p->A::doSth() ; // error: ambiguous
    p->B::doSth() ; // ok
    p->C::doSth() ; // ok
    // ...
}
```

Virtual base class

```
class A { ... };  
class B: public virtual A  
{ ... };  
class C: public virtual A  
{ ... };  
class D: public B, public C  
{ ... };
```

□ D has only 1 **class A**